Katholieke
Universiteit
Leuven

**Department of
Computer Science**

A Reverse-Hoops-Playing Robot Arm

Robotics (B-KUL-H02A4A)

Enmin Lin (r0927480), Kaixi Yao (r0926887)

Academic year 2025–2026

Reverse-Hoops-Playing Robot Arm

System Overview and Experimental Setup

The project defines a reverse-hoops task with noisy ball measurements, a constrained UR10, and a strict pass-through requirement [1]. This task admits a natural decomposition into estimation, feasibility-aware candidate selection, and constrained motion planning. The load-bearing design choice is feasibility-aware candidate selection: the rest of the report tests this claim by holding the Kalman estimator and NLP controller fixed while changing the selection rule. A better estimator alone does not solve the task, and a simple geometric catch point does not give a robot-executable catch.

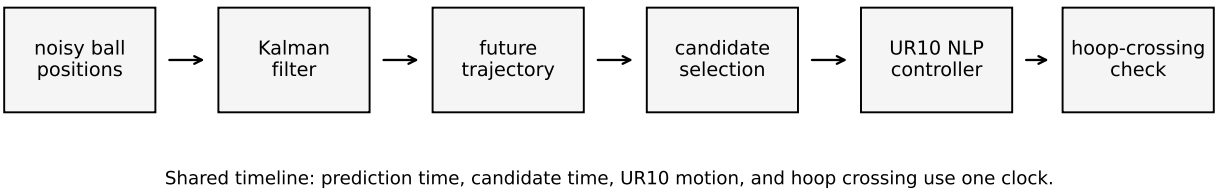

Figure 1: System pipeline used in the benchmark. The same ball prediction, candidate pool, NLP controller, and hoop-crossing metric support all strategy comparisons.

Table 1: Constants and modelling choices used by the estimator, controller, and success metric.

Item	Value	Quantitative role
Ball and hoop radii	0.12, 0.15 m	$r_h - r_b = 30$ mm radial tolerance
Measurement noise	1 mm Gaussian position noise	$R = \sigma^2 I_3$ in the Kalman update
Robot base	Fixed	Removes mobile-base planning from the report
Acceleration limit	$2\,\mathrm{rad/s^2}$	$\lvert\ddot{q}_{k,j}\rvert \le 2$ for every knot and joint
Hoop orientation	Top side never faces ground	TCP z-axis satisfies $R_{zz}(q_k) \ge 0$
Table clearance	Hoop center remains above table	$z_{\mathrm{tcp}}(q_k) \ge h_{\mathrm{table}} + m_{\mathrm{tcp}}$, hard path constraint
Self-collision proxy	Link-sphere clearance check	Positive sphere clearance means no proxy overlap after planning

The controller plans once after candidate selection and then executes the precomputed joint trajectory open-loop; no MPC runs inside the simulation. The benchmark therefore separates planner quality from execution quality: terminal TCP error measures the NLP fit to the predicted target, while radial crossing error measures the final open-loop result against the true simulated ball trajectory. The simulator includes a bounce model, but all reported catches occur before bounce. A post-bounce catch would require a switching model or an interacting multiple-model filter.

Task 1: Trajectory Estimation under Position Noise

The ball state uses position and velocity,

$$x_t = \begin{bmatrix} p_x & p_y & p_z & v_x & v_y & v_z \end{bmatrix}^\top . \tag{1}$$

The process model integrates constant horizontal velocity and known gravity in z,

$$x_{t+\Delta t} = Fx_t + Bg + w_t, \qquad y_t = Hx_t + v_t, \qquad v_t \sim \mathcal{N}(0, \sigma^2 I_3). \tag{2}$$

Here F is the constant-velocity transition matrix, Bg inserts gravitational acceleration, H selects position, and y_t is the measured ball position. The project gives independent Gaussian position noise, and the ball model is linear between bounces once gravity is known.

Filter choice. These assumptions match a Kalman filter: a Gaussian prior, linear dynamics, and linear Gaussian observations give the Bayes-optimal MMSE update and an analytic covariance [2]. This covariance is useful because it also tells whether prediction uncertainty grows to the same scale as the 30 mm hoop clearance. Both the dynamics and the measurement model are linear, so an EKF reduces to the same Kalman update. A particle filter would represent the same unimodal Gaussian belief with samples, so it would add sampling variance without improving the model fit.

The filter initializes from the first noisy position measurement with zero velocity and a broad velocity covariance. The robot joint state remains perfectly observable in the simulator, so the estimator only tracks the ball. Across 50 seeds at the project noise level, raw measurement RMSE was 1.74 mm, filtered RMSE was 0.90 mm, and 0.5 s future-prediction RMSE was 3.51 mm. The 95th percentile future-prediction RMSE was 6.35 mm, well below the 30 mm hoop clearance.

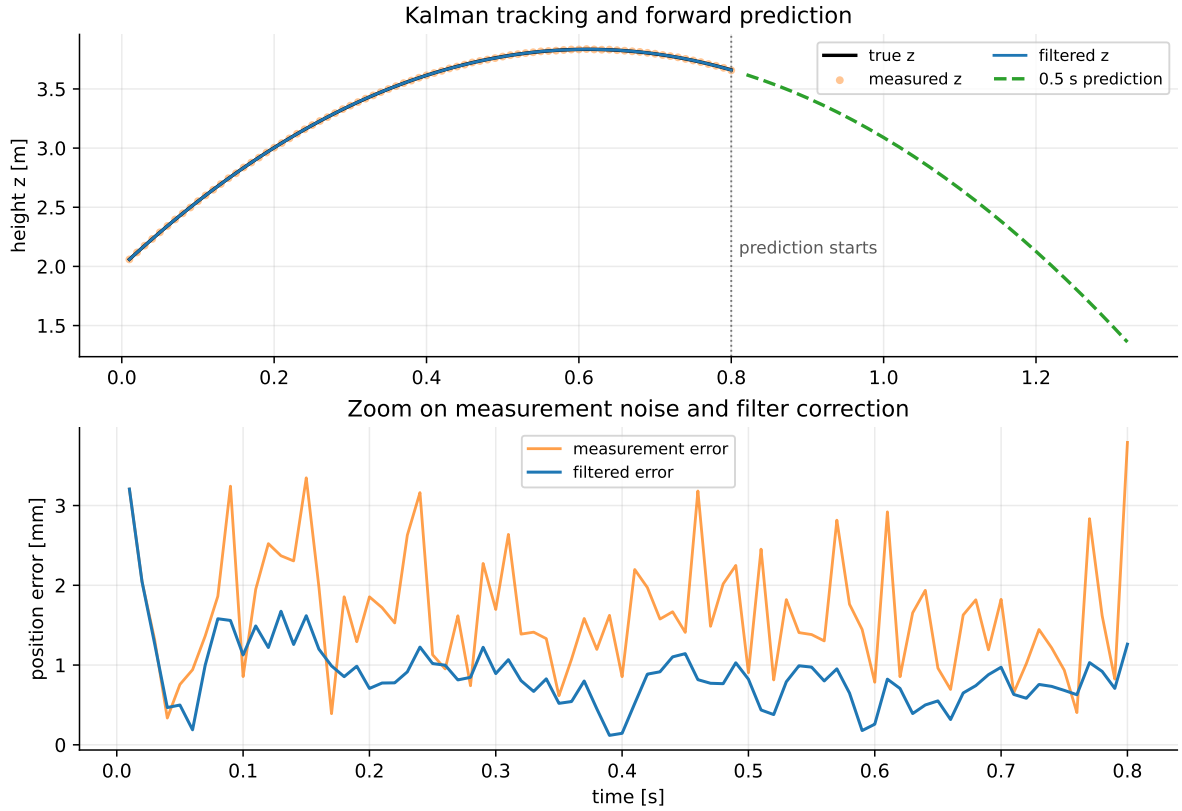

Figure 2: Kalman-filter prediction for one seed. The lower panel shows that filtering removes most of the 1 mm measurement jitter, and the 50-seed validation keeps the 0.5 s prediction error below 6.35 mm at the 95th percentile.

A measurement-noise sweep checks whether this filter choice is fragile. When noise increases from 1 to 10 mm, mean 0.5 s prediction RMSE increases from 3.67 to 10.34 mm, and the maximum observed RMSE remains 20.01 mm. The sweep uses an independent seed set, so the small difference between the 3.51 mm benchmark value and the 3.67 mm sweep value is sampling variance. These errors remain below the 30 mm radial clearance, so the stated 1 mm noise level does not make estimation the limiting part of the system.

Task 2: Geometric Candidate Selection Baselines

Task 2 asks for a simple interception point. The geometric baseline samples the predicted trajectory and rejects points less than 0.12 s ahead, below $z = 0.35$ m, or outside a workspace radius from the current hoop center. The standalone Task 2 validation uses 0.85 m, a tighter reachability screen from the Task 3 tests; the Task 4 benchmark uses 1.15 m to widen the shared candidate pool. That wider cap admits earlier attempts, so the 50-seed simple-geometric mean of 1.295 s is earlier than the standalone 1.36 s pick. For the validation seed, the rule selected position $[1.224, -0.119, 1.074]$ m.

This rule answers where the ball could be caught in Cartesian space. It does not answer whether the UR10 can put the hoop there with the required pose and acceleration. Across 50 seeds, the simple geometric rule attempted an early catch at 1.295 ± 0.012 s, but it produced 0 successful catches under the full NLP and hoop-crossing check.

A second baseline removes prediction completely. It uses a damped Jacobian pseudo-inverse Cartesian velocity controller,

$$\dot{q} = J(q)^\top \left(J(q)J(q)^\top + \lambda^2 I \right)^{-1} K(p_{\text{ball,meas}} - p_{\text{tcp}}), \tag{3}$$

with proportional gain K, damping λ, and velocity and acceleration saturation. This reactive controller also gives 0% success. Its closest TCP-to-ball distance was 0.176 ± 0.058 m, and even the best seed stayed 59.4 mm away, about twice the 30 mm radial tolerance. The controller reaches only the geometric closest-approach point of the flight path; it cannot accelerate the hoop fast enough to create a time-aligned catch. Prediction is therefore necessary, and a prediction-only geometric catch point remains insufficient.

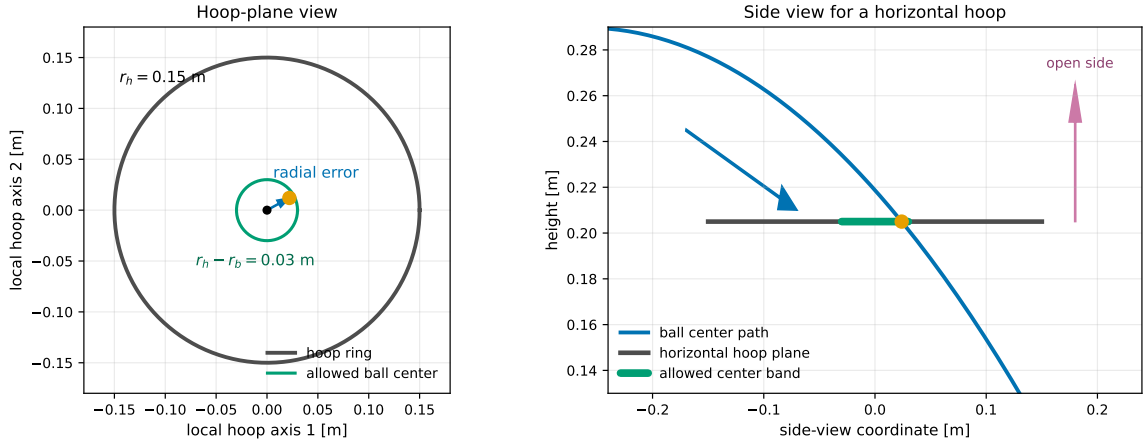

Figure 3: Success metric used in Task 3 and Task 4. The ball center must cross the hoop plane from the open side and lie within the $r_h - r_b = 30\,\text{mm}$ radial band; otherwise the ball touches the ring or misses the opening.

Task 3: Constrained Trajectory Optimization for the UR10

The controller solves a finite-horizon nonlinear program over joint position, velocity, and acceleration,

$$\{q_k, \dot{q}_k\}_{k=0}^{N}, \quad \{\ddot{q}_k\}_{k=0}^{N-1}. \tag{4}$$

It uses the second-order discrete dynamics

$$q_{k+1} = q_k + \dot{q}_k \Delta t + \tfrac{1}{2}\ddot{q}_k \Delta t^2, \qquad \dot{q}_{k+1} = \dot{q}_k + \ddot{q}_k \Delta t. \tag{5}$$

The optimization problem is

$$\begin{aligned} \min_{q,\dot{q},\ddot{q}} \; & w_p \|p_{\text{tcp}}(q_N) - p^\star\|_2^2 + \sum_{k=0}^{N-1} w_u \|\ddot{q}_k\|_2^2 + \sum_{k=1}^{N} w_v \|\dot{q}_k\|_2^2 \\ & + w_o (1 - R_{zz}(q_N))^2 + w_n \big(1 - (n_{\text{hoop}}(q_N)^\top \hat{v}_{\text{ball}})^2\big) \\ \text{s.t. } & q_0 = q_{\text{init}}, \quad \dot{q}_0 = \dot{q}_{\text{init}}, \\ & q_{\min} \le q_k \le q_{\max}, \quad -\dot{q}_{\max} \le \dot{q}_k \le \dot{q}_{\max}, \quad -\ddot{q}_{\max} \le \ddot{q}_k \le \ddot{q}_{\max}, \\ & z_{\text{tcp}}(q_k) \ge h_{\text{table}} + m_{\text{tcp}}, \quad R_{zz}(q_k) \ge 0, \qquad \forall k. \end{aligned} \tag{6}$$

Equation 5 keeps the planned joint trajectory dynamically consistent at the chosen discretization. In Eq. 6, $p^\star$ is the predicted ball position at the selected catch time, p_{tcp} is the hoop-center position, n_{hoop} is the hoop normal, and $\hat{v}_{\text{ball}}$ is the unit ball velocity at the catch point. The first cost term drives the hoop center to the ball. The next two terms reduce joint acceleration effort and joint velocity. The last two terms keep the hoop upright and align its opening with the ball direction. The squared dot product makes the alignment cost orientation-symmetric; the $R_{zz} \ge 0$ path constraint selects the upright branch. The inequalities enforce joint position, velocity, acceleration, table-clearance, and top-side orientation limits.

The validation uses $N = 28$, $\Delta t = 0.05\,\text{s}$, $w_p = 1500$, $w_u = 2 \times 10^{-2}$, $w_v = 10^{-3}$, $w_o = 10$, and $w_n = 5$. This gives a 1.4 s validation horizon; benchmark horizons are recomputed from candidate catch times. In the URDF, the fixed tool-to-TCP transform maps the physical ring normal to the TCP z-axis. The controller therefore uses TCP z as n_{hoop}. The scalar $R_{zz} = e_z^\top R_{\text{tcp}}(q) e_z$ is the world-z component of that TCP z-axis, so $R_{zz}(q_k) \ge 0$ enforces that the top side never points below the horizontal plane. The hoop opens along $+n_{\text{hoop}}$, so a successful crossing requires both the correct side-of-plane sign and radial-band membership. Pinocchio supplies URDF forward kinematics [5]. CasADi builds the symbolic NLP [4], and IPOPT solves it [3].

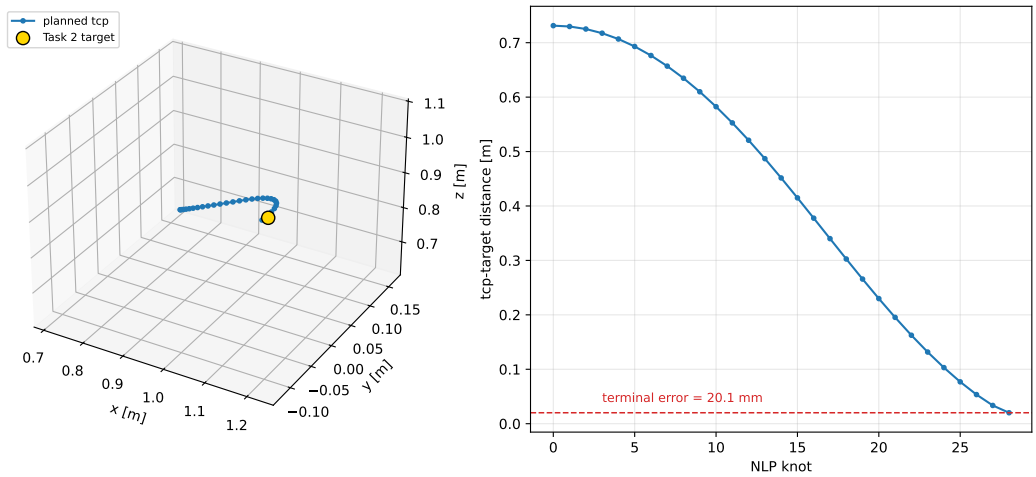

Figure 4: Task 3 finite-horizon NLP result. The hoop center reaches 20.1 mm terminal TCP error while the path constraints keep the motion executable.

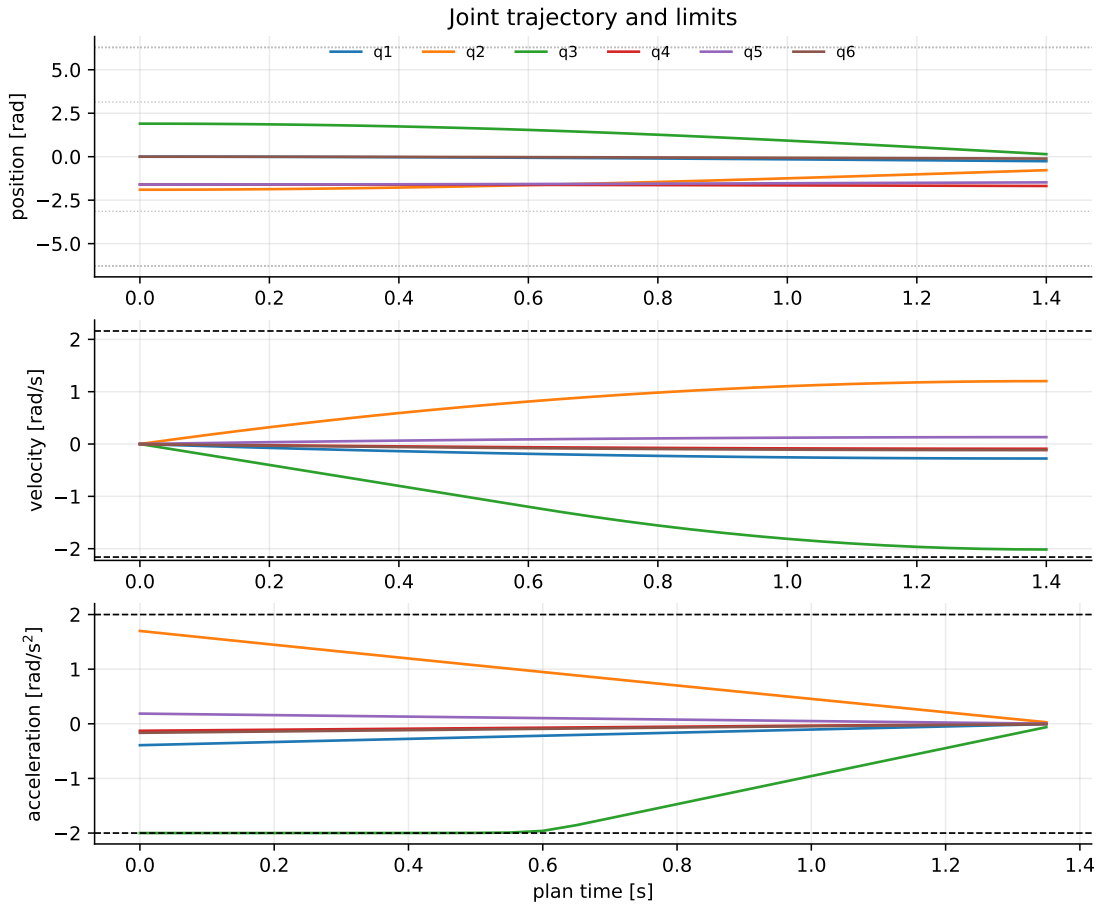

Figure 5: Joint profiles for the same plan. All accelerations approach but do not exceed $\pm 2\,\mathrm{rad/s^2}$, and joint 2 reaches the highest velocity utilization at 64% of its limit.

Figure 4 shows the spatial effect of the NLP: the hoop center moves from a visibly unreachable initial offset to a 20.1 mm terminal error at the selected catch point. Figure 5 shows the limiting mechanism. The robot is not primarily short of velocity; joint 2 uses only 64% of its velocity bound. The active limitation is acceleration, with the acceleration profile touching the $\pm 2\,\mathrm{rad/s^2}$ bound.

For the Task 2 target, the NLP reduced the hoop-target distance from 0.731 m to 0.020 m. It solved in 0.071 s and 15 IPOPT iterations on the validation run. The maximum acceleration was $1.9996\,\mathrm{rad/s^2}$, the maximum velocity ratio was 0.640, and the minimum R_{zz} value was 0.732. The minimum TCP-table clearance was 0.622 m, the minimum frame-table clearance was 0.115 m, and the minimum self-collision clearance proxy was 0.163 m. Table clearance and hoop orientation enter the NLP as hard path constraints.

Exact mesh self-collision would add many non-smooth distance constraints, so the report uses a conservative link-sphere clearance check after planning. All reported successful plans keep positive sphere clearance. A stricter future version could add capsule or sphere signed-distance constraints directly to Eq. 6.

The w_o ablation checks whether the upright-orientation soft cost drives the result or merely regularizes it. Across $w_o \in$

$\{0, 1, 10, 50\}$, time-first benchmark success stayed at 98%. This shows that the hard $R_{zz} \geq 0$ path constraint and the pose-aware normal term set the final geometry; the terminal R_{zz} cost mainly regularizes the pose.

Task 4: Feasibility-Aware Candidate Selection

Task 4 asks for higher success rate and shorter interception time. The result depends on the comparison domain. The simple geometric rule attempts an earlier catch, but it has 0% success, so its attempted time is not a successful-interception baseline. The fair time comparison uses successful feasible policies evaluated on the same candidate pool.

The selector forms a shared candidate pool from the predicted trajectory, solves the Task 3 NLP for each shortlisted candidate, and accepts only candidates that pass three checks: NLP convergence, hoop-plane crossing by the true ball trajectory, and zero safety penalty. All 396 evaluated candidate NLPs converged, but only 164 candidates passed the hoop-crossing test. Each seed therefore produced 7.9 evaluated candidates and 3.3 feasible catches on average. The feasibility filter rejects about 59% of geometrically plausible candidates. This split identifies the bottleneck: the robot can solve the motion plans, but many candidate times do not place the hoop in a crossing-compatible pose. Compared with the simple geometric attempt at 1.295 s, the time-first feasible selector catches 92 ms later. That number is a catch-time delay, not computation time. The delay is about two control ticks because the NLP uses $\Delta t = 50$ ms, and it raises success from 0% to 98%.

The balanced selector ranks feasible candidates by a weighted sum of catch time, terminal TCP error, acceleration use, velocity use, and safety penalty, with weights $15.0, 100.0, 0.20, 0.20, 50.0$. Time-first and late-feasible policies use the same candidate pool but select the earliest or latest successful candidate directly. Figure 6 visualizes this split: red marks the simple geometric pick, and yellow marks the feasibility-aware catch that the UR10 can actually cross.

Both the 1–10 mm noise sweep and the covariance-rejection ablation reinforce the same conclusion: the bottleneck is geometric feasibility, not estimator quality. Over 20 downstream seeds, time-first success stayed at 95% for all tested noise levels. With 20 seeds, one failure costs 5%, so this result is consistent with the 50-seed 98% benchmark. A candidate-rejection ablation that drops feasible candidates above a conservative 0.18 m forward-covariance threshold selected the same catches as time-first feasible across all 50 seeds. These checks keep the covariance output as a useful diagnostic, but they do not make it the decisive selector in this open-loop benchmark.

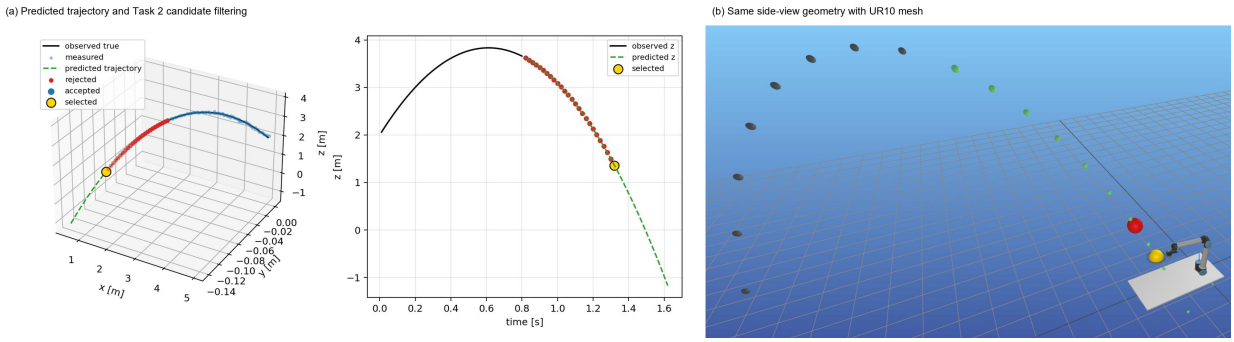

Figure 6: Task 2 and Task 4 candidate geometry in matching views. The left panels show predicted-trajectory filtering for the candidate pool; the right panel renders the same scene in Meshcat. Colors: grey = observed past trajectory, green = predicted future trajectory, red = simple geometric candidate, yellow = selected feasible catch.

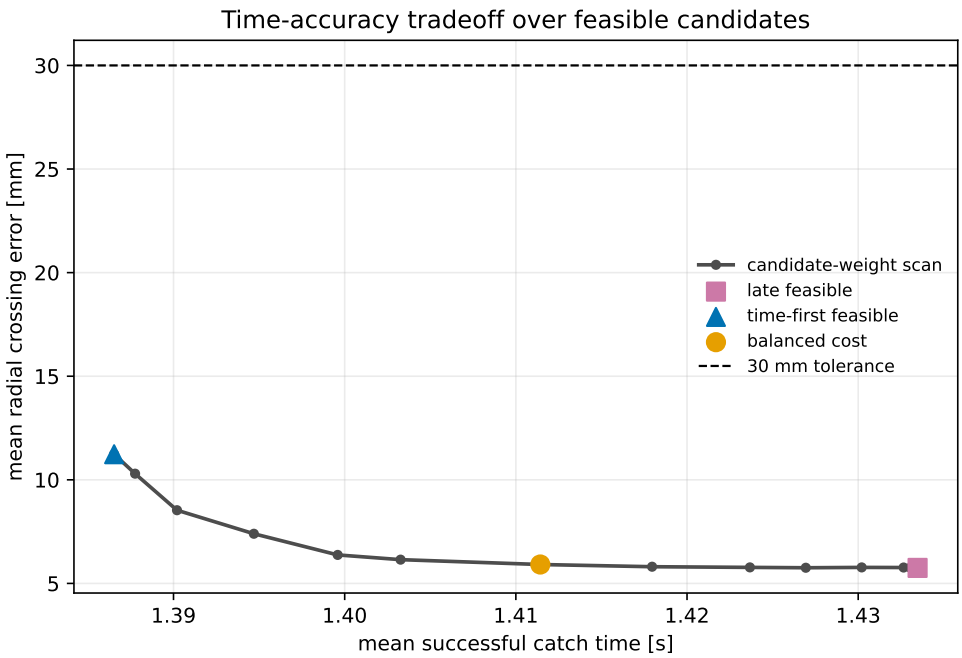

Figure 7: Empirical candidate-weight scan on the time-accuracy plane. Time-first gives the early-time end, late feasible gives the accurate but slower end, and the elbow near balanced cost buys roughly 5 mm of radial accuracy for about 25 ms.

Table 2: Final 50-seed benchmark. Time is catch time from ball release. TCP is terminal hoop-center error to the predicted target. Radial is true hoop-plane crossing error. Max $\ddot{q}$ is the largest absolute joint acceleration in rad/s^2, with hard bound 2. The asterisked simple time is attempted, not successful.

Strategy	Success	Time [s]	TCP [mm]	Radial [mm]	Max $\ddot{q}$	IPOPT
Reactive no prediction	0%	–	–	–	–	–
Simple geometric	0%	1.295*	–	–	–	–
Late feasible	98%	1.433 ± 0.009	1.87	5.75	1.65	10.4 it.
Time-first feasible	98%	1.387 ± 0.018	13.18	11.24	2.00	13.5 it.
Balanced cost	98%	1.411 ± 0.015	3.22	5.91	1.81	12.3 it.

The time-first feasible policy gives the strongest answer to the Task 4 wording. It raises full-system success from 0% to 98% and catches 0.025 s earlier than balanced cost and 0.047 s earlier than late feasible. The latter gap is about one control tick. Late feasible gives the best accuracy, while balanced cost lands between the two extremes. The submitted Task 4 answer is time-first because the wording explicitly asks for shorter interception time. If the objective weighted hoop-crossing accuracy more heavily, balanced cost or late feasible would be the stronger engineering choice. Figure 7 makes the time-accuracy tradeoff explicit instead of collapsing it into one success-rate number.

Time-first selects the earliest feasible candidate, so it runs with the tightest motion budget. The NLP saturates the acceleration bound and has less freedom to refine pose. Balanced and late-feasible candidates occur later, where the NLP has slack to align the hoop normal with the ball direction. This explains the lower TCP and radial errors in those rows. The TCP column reports hoop-to-target distance at the NLP terminal knot, while the radial column reports hoop-to-ball distance when the ball actually crosses the hoop plane. With time-first, the hoop can sweep past the ball, so the crossing instant can be closer than the terminal knot. Late feasible drives TCP error below 2 mm. Radial error stays near 6 mm because the true ball crosses the hoop plane a few millimetres off the predicted target, a residual of measurement noise and candidate-time discretization.

The open-loop model-execution gap remains small. For successful time-first catches, the NLP terminal error to the predicted target averaged 13.18 mm, the predicted hoop-crossing radial error averaged 9.36 mm, and the true-ball radial error averaged 11.24 mm. The predicted and true radial means differ by 1.87 mm ($11.24 - 9.36$), while the per-seed mean absolute predicted-to-true radial gap is 4.41 mm. This separates solver quality from execution quality without hiding the open-loop assumption.

Failure Analysis and Interpretation

The only time-first failure is seed 11. Figure 8 places it near the 95th-percentile line of the 50-seed v_y distribution. After fixing the hoop normal to the TCP z-axis in the Task 3 setup, seed 11's best radial error dropped from 145.2 to 51.5 mm, but this still exceeds the 30 mm tolerance. The same correction recovered seed 41, which now crosses at 24.7 mm radial error.

The seven shortlisted seed 11 candidates all solved the NLP and all crossed the hoop plane, but all crossed outside the radial tolerance. Their radial errors decrease from 758 to 51.5 mm as catch time moves from 1.32 to 1.44 s. The 758 mm worst case occurs at the earliest candidate, where the UR10 cannot physically reach the predicted target in time. Each candidate reaches the 2 rad/s^2 acceleration bound: early candidates saturate joint 3, and later candidates saturate joint 2. The failure is therefore acceleration-limited geometry, not solver failure or estimator divergence.

A mitigation run with 16 candidates and a 1.35 m workspace radius (default: 8 candidates and 1.0 m) did not recover seed 11. Recovering it would require a longer prediction horizon or a different candidate sampler.

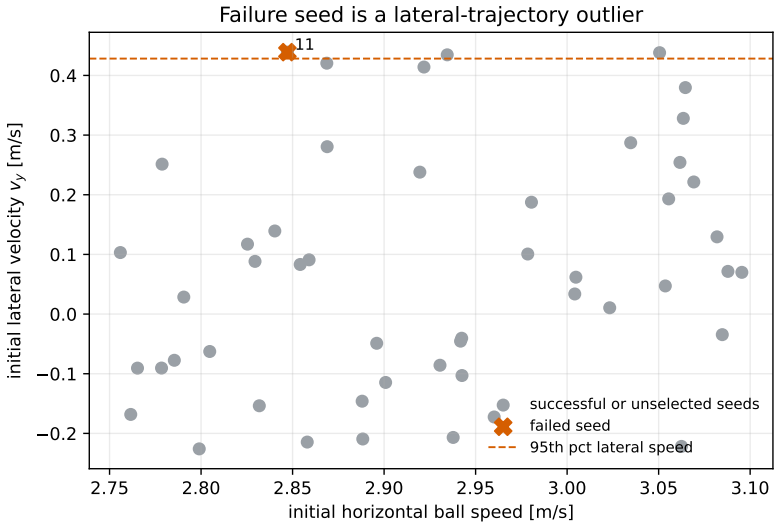

Figure 8: Failure mechanism for seed 11. The red marker is the failed seed and the grey markers are the other seeds; the dashed line marks the empirical 95th percentile of lateral velocity. Seed 11 sits near this high-v_y boundary and fails by acceleration-limited geometry, not estimator divergence or NLP failure.

The final architecture uses a direct-transcription optimal control problem with hard physical constraints and soft secondary objectives. This choice matters because a single-step Jacobian controller can chase a Cartesian target but cannot reserve

acceleration over a future window. A closed-form smooth trajectory can reduce jerk, but it does not enforce hoop crossing, joint limits, and top-side orientation at the same time. The horizon NLP is the smallest formulation in this report that handles those requirements together.

Conclusion

The experiments support the thesis stated in the setup. Prediction is necessary, because the reactive controller never reaches the ball within the tolerance. Prediction alone is insufficient, because the simple geometric selector also gives 0% success. The decisive step is feasibility-aware candidate selection: it keeps the estimator and controller fixed, filters candidates through the UR10 NLP and the true hoop-crossing geometry, and raises success to 98%. Time-first feasible gives the shortest successful catch among the evaluated policies, while the balanced selector shows the measurable cost of buying a few millimetres of radial accuracy.

References

[1] Robotics course project description, *A Reverse-Hoops-Playing Robot Arm*, KU Leuven, 2025–2026.

[2] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, 82(1), 35–45, 1960. DOI: 10.1115/1.3662552.

[3] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, 106, 25–57, 2006. DOI: 10.1007/s10107-004-0559-y.

[4] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi: a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, 11, 1–36, 2019. DOI: 10.1007/s12532-018-0139-4.

[5] J. Carpentier et al., "The Pinocchio C++ library: a fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives," IEEE/SICE International Symposium on System Integration, 2019.